

Projeto Prático #3: O projeto deve ser desenvolvido por grupos de *no máximo* três alunos. É estritamente proibida a partilha de código entre alunos de grupos diferentes.

Plágio: Além de inspeção manual, será também executado um software de deteção de plágio no código-fonte do projeto. *Todos* os alunos que submeterem código obtido através de plágio terão os seus projetos anulados.

Uso de Inteligência Artificial: É permitido utilizar ferramentas de IA. No entanto, todos os membros do grupo devem ser capazes de compreender e *explicar* o projeto *na sua totalidade* e, em especial, a arquitetura de microserviços e padrões distribuídos desenvolvidos.

Submissão: A data de submissão do projeto é **03/06/2026** às 23:59. A descrição do processo de submissão pode ser encontrada no enunciado.

TascaEats: Delivery de Comida

1 Contextualização

Com o sucesso da implementação nas fases anteriores, a aplicação **TascaEats** sofreu um aumento exponencial de tráfego. A consultoria Compadres Sempre Saciados percebeu que não seria mais viável manter o sistema num monólito, pois começou a ter problemas com a escalabilidade de certas operações críticas. Como tal, foi decidido realizar uma migração para uma arquitetura de microserviços, começando pela gestão de entregadores.

O objetivo da Fase 3 é aplicar os princípios de *Domain-Driven Design* para extrair o *Bounded Context* de **Gestão de Entregas** para um Microserviço. Todo o ecossistema será orquestrado localmente utilizando Docker Compose¹.

2 Casos de Uso e Validação

Os casos de uso permanecem os mesmos em relação à Fase 2. No entanto, os casos de uso relacionados com a gestão de entregas (e.g., atribuição de entregadores, atualização de estado) devem agora ser implementados exclusivamente no novo microserviço.

Nenhuma das funcionalidades da segunda entrega deve ser perdida ou ter a sua experiência de utilizador degradada. O desafio é garantir que, apesar da nova arquitetura distribuída, o sistema continue a funcionar de forma fluida e transparente para o cliente final. As interfaces gráficas continuam a chamar os endpoints no Monólito, mas este já não processa a lógica de negócio das entregas. Para essas funcionalidades, o Monólito passa a atuar apenas como um produtor de eventos para o Kafka, delegando o processamento para o Microserviço.

¹Não utilizaremos infraestrutura de Cloud ou Kubernetes nesta fase.

3 Extração do Domínio de Entregas

O domínio escolhido para extração é o de **Gestão de Entregas**. Este domínio envolve os entregadores, a atribuição de pedidos e a atualização de estados.

As seguintes funcionalidades devem ser feitas do novo Microserviço **Gestão de Entregas**:

1. Registo, atualização e gestão do perfil de Entregadores.
2. Filtros e buscas relativos a entregadores.
3. Atribuição (manual ou automática) de um entregador a um pedido recém-pago.
4. Atualização do estado da entrega (e.g., “A caminho”, “Entregue”).

Reparem que o microserviço de entregas não tem conhecimento direto do cliente ou do restaurante. Ele apenas recebe referências lógicas (e.g., `pedidoId`, `morada`, ...) e é responsável por gerir a lógica de gestão de entregas. O monólito continua a gerir o restante da aplicação. Isso inclui a comunicação com as interfaces gráficas (Web e JavaFX) que devem funcionar corretamente através da consistência eventual do sistema.

4 Mudança Arquitetural

A arquitetura do vosso projeto deve agora suportar os seguintes paradigmas e padrões de estruturação:

1. As interfaces Web e JavaFX **não** devem comunicar diretamente com o monólito. É necessário configurar um **Nginx** no `docker-compose` a atuar como *Reverse Proxy*. O **Nginx** receberá todos os pedidos (REST da Web e gRPC do JavaFX) e irá encaminhá-los para o Monólito, que continua a atuar como o ponto central de entrada da aplicação.
2. O Monólito e o novo Microserviço de **Gestão de Entregas** devem comunicar *exclusivamente* de forma assíncrona utilizando eventos através do Apache Kafka.
3. Devem garantir a consistência eventual².
4. O microserviço deve ter a sua própria base de dados separada. O monólito não pode aceder à base de dados do microserviço e vice-versa.

Estas mudanças irão implicar uma reestruturação significativa do código. A criação da base de dados separada, a configuração do Nginx, a orquestração dos serviços e a implementação de forma a garantir consistência eventual são parte integrante desta fase e serão avaliadas rigorosamente.

5 Como Entregar

Para entregar o trabalho, basta criar uma tag chamada *fase3* e enviá-la para o repositório. O repositório deverá conter todo o código-fonte (separado logicamente e em diferentes pastas para cada serviço), o ficheiro `docker-compose.yml` e as configurações de infraestrutura (e.g., ficheiro `nginx.conf`). Não se esqueça de incluir o link do vídeo de demonstração no ficheiro `README.md`.

²Sagas são uma excelente opção para lidar com transações distribuídas

```
git tag fase3
git push origin fase3
```

O projeto **obrigatoriamente** deverá correr num ambiente Docker. A execução do comando `docker-compose up` deve ser suficiente para iniciar toda a infraestrutura (Bases de Dados, Kafka, Monólito, Microserviço e Nginx) sem erros. Falhas em **EXECUTAR** o projeto pela equipa docente resultarão em **grave penalização**.

6 Critérios de Avaliação

Esta entrega corresponde aos 30% finais da nota do projeto (sendo os 70% iniciais referentes às Fases 1 e 2). Os critérios focam-se fortemente nos padrões arquiteturais introduzidos:

6.1 Arquitetural

- Isolamento e criação de uma base de dados exclusiva e funcional para o microserviço.
- Configuração correta do Nginx a atuar como *Reverse Proxy* e responsável pelo encaminhamento de pedidos REST e gRPC.
- Uso correto do Kafka para comunicação orientada a eventos.
- Implementação que garanta a consistência eventual do sistema.

6.2 Demonstração

- Qualidade e robustez do ficheiro `docker-compose.yml`.
- Clareza no vídeo de demonstração: deve ficar claro o caminho de um pedido a passar pelo Nginx, gerar eventos no Kafka, ser processado no microserviço, e refletir a atualização na base de dados isolada³.
- Granularidade, frequência e divisão equitativa dos *commits* entre os membros do grupo.

Nota Final: Faça um esboço inicial das arquiteturas envolvidas. Planeiem com antecedência as mensagens do Kafka e a estrutura do Docker. Utilizem o fórum de discussão no Moodle para dúvidas arquiteturais. Bom trabalho!

³Pode ser necessário revisitar alguns dos casos de uso anteriores (os que envolvem a integração com o microserviço). Não é necessário demonstrar todos os casos de uso do projeto.